

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Отчет о выполнении лабораторной работы №1
по дисциплине «Теория алгоритмов»
«Реализация двумерного клеточного автомата»
Вариант №10

Студент,
группы 5130201/30003

_____ Мартыненко А. П.

Преподаватель

_____ Востров А. В.

«_____» _____ 2025г.

Содержание

Введение	3
1 Математическое описание	4
1.1 Понятие клеточного автомата	4
1.2 Функция	5
1.3 Граничные условия	6
1.4 Классификация динамического поведения	8
2 Особенности реализации	9
2.1 Логическая часть	9
2.2 Графическая часть	10
3 Анализ клеточного автомата	13
4 Результаты работы программы	15
Заключение	18
Список использованной литературы	19
Приложение А «Графическая часть приложения»	20

Введение

В лабораторной работе необходимо:

Реализовать двумерный клеточный автомат с окрестностью фон Неймана в соответствии с полученным номером №, который считается по формуле

$$\text{№} = \text{номер варианта} \times 11 \times \text{год рождения} \times \text{день} \times \text{месяц}$$

(начальные условия в формате «день / месяц / год»: «1 / 6 / 2005»).

Граничные условия - торроидальные. Пользователь определяет ширину поля и количество итераций. Необходимо учесть возможность ввода различных начальных условий (как вручную, так и случайным образом) по выбору пользователя. Реализация возможна в консоли или в графике. В итоге необходимо проанализировать свой клеточный автомат в отчете (его поведение, паттерны, "сходимость" и т.д.).

1 Математическое описание

1.1 Понятие клеточного автомата

Клеточный автомат - это дискретная динамическая система, состоящая из совокупности одинаковых клеток, соединенных между собой одинаковым способом. Каждая клетка находится в одном из конечного множества состояний.

Элементы клеточного автомата:

1. Решетка - одномерная, двумерная или многомерная сетка клеток C .

Решетка является однородной: невозможно отличить никакие два места по ландшафту. В данной лабораторной работе используется двумерная (квадратная) решетка, где каждая клетка имеет свои координаты.

2. Состояние клетки и правила перехода

Каждая клетка $C_{(i,j)}$ может находиться в одном из в одном из конечного множества состояний $S = s_1, s_2, \dots, s_k$. Состояние каждой клетки обновляется на основе набора правил, которые применяются к текущему состоянию клетки и состояниям соседних для неё клеток. Правила определяют, как должно измениться состояние клетки в следующий момент времени на основе текущего состояния клетки и состояний её соседей. Изменения значений всех клеток происходят одновременно после вычисления нового состояния каждой клетки решетки.

3. Инициализация

Начальное состояние клеточного автомата описывается функцией $f: L \rightarrow S$, где L — решетка, S — множество состояний. Функция присваивает каждой клетке начальное состояние либо по вводу пользователя, либо случайным образом.

4. Окрестность

В данной лабораторной работе используется окрестность фон Неймана - совокупность ячеек в сетке имеющих общую сторону с данной ячейкой. Соответственно, у каждой клетки $C_{(i,j)}$ есть четыре соседа (рисунок 1)

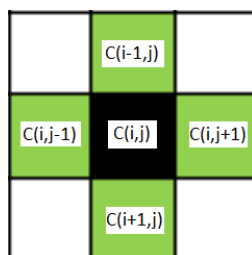


Рис. 1. Окрестность фон Неймана

1.2 Функция

Функция перехода клеточного автомата имеет вид $func : S^{n+1} \rightarrow S$, где n — размер окрестности, включая саму клетку $C(i, j)$, а S^{n+1} — множество всех возможных конфигураций состояний в этой окрестности.

Эволюция системы происходит в дискретные моменты времени. На каждом шаге t состояние клетки обновляется согласно правилу:

$$C(i, j)^{(t+1)} = f \left(C_{(i,j-1)}^{(t)}, C_{(i-1,j)}^{(t)}, C_{(i,j+1)}^{(t)}, C_{(i+1,j)}^{(t)}, C_{(i,j)}^{(t)} \right), \text{ где}$$

$\forall i, j \in [0, n-1], t \in [0, it-1]$, n - размер поля, it - количество итераций

Данная функция является функцией пяти переменных, так как окрестность фон Неймана для двумерного автомата включает 4 соседние клетки и саму центральную клетку, что в сумме составляет 5 аргументов функции.

Вектор значений функции считается по формуле:

$$f = \text{номер варианта} \times 11 \times \text{год рождения} \times \text{день} \times \text{месяц}$$

$$f = 10 \times 11 \times 2005 \times 1 \times 6 = 1323300_{10} = 101000011000100100100_2$$

Дополним полученный вектор функции незначащими нулями в начале, чтобы получить вектор нужной размерности (для функции пяти переменных - 32 значения). Итоговая таблица истинности представлена в таблице 1.

Таблица 1. Таблица истинности функции f .

a	b	c	d	e	$f(a, b, c, d, e)$
0	0	0	0	0	0
0	0	0	0	1	0
0	0	0	1	0	0
0	0	0	1	1	0
0	0	1	0	0	0
0	0	1	0	1	0
0	0	1	1	0	0
0	0	1	1	1	0
0	1	0	0	0	0
0	1	0	0	1	0

a	b	c	d	e	$f(a, b, c, d, e)$
0	1	0	1	0	0
0	1	0	1	1	1
0	1	1	0	0	0
0	1	1	0	1	1
0	1	1	1	0	0
0	1	1	1	1	0
1	0	0	0	0	0
1	0	0	0	1	0
1	0	0	1	0	1
1	0	0	1	1	1
1	0	1	0	0	0
1	0	1	0	1	0
1	0	1	1	0	0
1	0	1	1	1	1
1	1	0	0	0	0
1	1	0	0	1	0
1	1	0	1	0	1
1	1	0	1	1	0
1	1	1	0	0	0
1	1	1	0	1	1
1	1	1	1	0	0
1	1	1	1	1	0

1.3 Граничные условия

По условию задания необходимо задать тороидальные граничные условия. Это означает, что решетка «замыкается» в тор, то есть верхняя граница соединяется с нижней, а левая - с правой.

Для поля размером $n \times n$ тороидальные условия задаются следующим образом:

- Для координаты i : $i_{\text{mod}} = (i + n) \bmod n$

- Для координаты j : $j_{\text{mod}} = (j + n) \bmod n$

Рассмотрим все случаи, затрагивающие граничные условия:

1. Левый верхний угол $(0, 0)$:

$$C(0, 0)^{(t+1)} = f \left(C_{(0, n-1)}^{(t)}, C_{(n-1, 0)}^{(t)}, C_{(0, 1)}^{(t)}, C_{(1, 0)}^{(t)}, C_{(0, 0)}^{(t)} \right)$$

2. Правый верхний угол $(0, n - 1)$:

$$C(0, n - 1)^{(t+1)} = f \left(C_{(0, n-2)}^{(t)}, C_{(n-1, n-1)}^{(t)}, C_{(0, 0)}^{(t)}, C_{(1, n-1)}^{(t)}, C_{(0, n-1)}^{(t)} \right)$$

3. Левый нижний угол $(n - 1, 0)$:

$$C(n - 1, 0)^{(t+1)} = f \left(C_{(n-1, n-1)}^{(t)}, C_{(n-2, 0)}^{(t)}, C_{(n-1, 1)}^{(t)}, C_{(0, 0)}^{(t)}, C_{(n-1, 0)}^{(t)} \right)$$

4. Правый нижний угол $(n - 1, n - 1)$:

$$C(n - 1, n - 1)^{(t+1)} = f \left(C_{(n-1, n-2)}^{(t)}, C_{(n-2, n-1)}^{(t)}, C_{(n-1, 0)}^{(t)}, C_{(0, n-1)}^{(t)}, C_{(n-1, n-1)}^{(t)} \right)$$

5. Верхний ряд (кроме угловых клеток)

$$C(0, j)^{(t+1)} = f \left(C_{(0, j-1)}^{(t)}, C_{(n-1, j)}^{(t)}, C_{(0, j+1)}^{(t)}, C_{(1, j)}^{(t)}, C_{(0, j)}^{(t)} \right)$$

6. Нижний ряд (кроме угловых клеток)

$$C(n - 1, j)^{(t+1)} = f \left(C_{(n-1, j-1)}^{(t)}, C_{(n-2, j)}^{(t)}, C_{(n-1, j+1)}^{(t)}, C_{(0, j)}^{(t)}, C_{(n-1, j)}^{(t)} \right)$$

7. Правый столбец (кроме угловых клеток)

$$C(i, n - 1)^{(t+1)} = f \left(C_{(i, n-2)}^{(t)}, C_{(i-1, n-1)}^{(t)}, C_{(i, 0)}^{(t)}, C_{(i+1, n-1)}^{(t)}, C_{(i, n-1)}^{(t)} \right)$$

8. Левый столбец (кроме угловых клеток)

$$C(i, 0)^{(t+1)} = f \left(C_{(i, n-1)}^{(t)}, C_{(i-1, 0)}^{(t)}, C_{(i, 1)}^{(t)}, C_{(i+1, 0)}^{(t)}, C_{(i, 0)}^{(t)} \right)$$

1.4 Классификация динамического поведения

Динамическое поведение - характер эволюции во времени, который определяется правилами перехода и начальными условиями. Оно описывает, как система развивается и эволюционирует от шага к шагу, формируя устойчивые, периодические или хаотические структуры.

Паттерн - устойчивые конфигурации клеток, возникающие в процессе эволюции. Паттерны могут быть статическими (неизменными), периодическими (повторяющимися через определённое число шагов) или динамическими (перемещающимися и взаимодействующими структурами).

Сходимость - свойство системы приходить к устойчивому состоянию или циклической конфигурации, которая перестаёт изменяться при дальнейших итерациях.

Классификация по типам поведения (по Вольфраму):

1. Класс I: полная сходимость к однородному состоянию.
Системы характеризуются тенденцией к единообразию и повторению. Они демонстрируют простые паттерны, которые быстро переходят в устойчивое состояние, где все клетки принимают однородное состояние — либо живые, либо мёртвые. Примеры: правило 0 и 255.
2. Класс II: сходимость к простым периодическим или статическим паттернам.
Системы демонстрируют простое, но при этом циклическое поведение. Они часто создают локализованные паттерны и изолированные области активности. Эти автоматы демонстрируют сочетание повторяющихся паттернов, что приводит к динамической эволюции. Примеры: правила 4, 108, 218 и 250.
3. Класс III: хаотическое поведение со сложными непериодическими паттернами.
Системы генерируют сложные и непериодические паттерны, чувствительные к начальному состоянию. Поведение выглядит случайным, хотя полностью определяется детерминированными правилами. Системы этого класса не сходятся и не повторяются, создавая постоянно меняющиеся структуры. Примеры: правило 30 и 150.
4. Класс IV: сложное поведение с возникающими структурами и возможностью универсальных вычислений.
Системы характеризуются случайным и непредсказуемым поведением. В них нет заметных закономерностей или регулярности, а их эволюция выглядит крайне неупорядоченной. Эти автоматы проявляют свойства, напоминающие случайность. Такие системы считаются наиболее интересными с точки зрения динамики, поскольку способны к универсальным вычислениям по Тьюрингу. Пример: правило 110.

2 Особенности реализации

2.1 Логическая часть

Класс `CellularAutomaton` инкапсулирует поведение двумерного клеточного автомата.

Константа `RULE_STR` описывает двоичное представление правила перехода конечного автомата. Каждый бит определяет новое состояние клетки для конкретной комбинации соседей. Правило задается битовой строкой длины 32 бита, так как в функции всего пять входов. Для корректной работы строка дополняется слева нулями до нужной длины.

Листинг 1. Работа с правилом перехода

```
1 RULE_STR = "101000011000100100100"
2 RULE_STR = RULE_STR.zfill(32)
3 rule = [int(b) for b in RULE_STR]
4 rule = [0] * (32 - len(rule)) + rule
5
```

Метод `__init__` предназначен для создания нового клеточного автомата заданного размера. Он создает двумерный массив заданной размерности и устанавливает флаг выполнения `running` в `False`.

Вход: размер поля, ссылка на инициализируемый объект.

Выход: инициализированный объект класса `CellularAutomaton`.

Листинг 2. Метод создания объекта

```
1 def __init__(self, n):
2     self.n = n
3     self.grid = [[0 for _ in range(n)] for _ in range(n)]
4     self.running = False
5
```

Метод `clear` предназначен для сброса поля к начальному состоянию, то есть заполнения его нулями.

Вход: ссылка на объект класса.

Выход: пустой объект класса `CellularAutomaton`.

Листинг 3. Метод очистки поля

```
1 def clear(self):
2     for r in range(self.n):
3         for c in range(self.n):
4             self.grid[r][c] = 0
5
```

Метод `randomize` предназначен для создания случайной начальной конфигурации. Для каждой ячейки списка он генерирует случайное число, и, если число меньше порогового значения, устанавливает ячейку в состояние 1.

Вход: ссылка на объект, пороговое значение.

Выход: модифицированный объект класса `CellularAutomaton`.

Листинг 4. Метод случайного заполнения

```

1 def randomize(self, p=0.3):
2 for r in range(self.n):
3 for c in range(self.n):
4 self.grid[r][c] = 1 if random.random() < p else 0
5

```

Метод `step` предназначен для выполнения одной итерации эволюции автомата. Для корректных вычислений создается новая пустая сетка того же размера, затем с помощью двойного цикла получаются значения соседей клетки и самой клетки. Для учета тороидальных границ используется остаток от деления. Полученные значения собираются в битовую маску, которая при обращении к глобальному списку `rule` получает новое состояние.

Вход: ссылка на объект.

Выход: объект класса `CellularAutomaton`.

Листинг 5. Метод эволюции автомата

```

1 def step(self):
2 new_grid = [[0 for _ in range(self.n)] for _ in range(self.n)]
3 for i in range(self.n):
4 for j in range(self.n):
5 a = self.grid[(i - 1) % self.n][j]
6 b = self.grid[i][(j + 1) % self.n]
7 c = self.grid[(i + 1) % self.n][j]
8 d = self.grid[i][(j - 1) % self.n]
9 e = self.grid[i][j]
10 idx = (a << 4) | (b << 3) | (c << 2) | (d << 1) | e
11 new_grid[i][j] = rule[idx]
12 self.grid = new_grid
13

```

2.2 Графическая часть

Для отрисовки используется стандартная библиотека Python - Tkinter. Это встроенная библиотека для создания графических интерфейсов, которая предоставляет разработчику простые средства для построения оконных приложений. Tkinter основан на кроссплатформенном фреймворке Tcl/Tk, благодаря чему приложения, написанные с его использованием, одинаково работают под Windows, macOS и Linux без необходимости установки сторонних зависимостей.

Класс `AutomatonApp` инкапсулирует управление графической частью приложения и взаимодействие пользователя с логической частью автомата, представленной классом `CellularAutomaton`. Полный код представлен в листинге 4.

При запуске программы создаётся основное окно, содержащее:

- панель управления, включающую поля ввода размера сетки и количества итераций;

- ползунок скорости, регулирующий задержку между итерациями (в миллисекундах);
- кнопки: «Запустить», «Пауза/Продолжить», «Случайно», «Очистить»;
- информационное поле с подсказкой по ручному вводу и отображением текущей итерации.

Для визуализации клеток используется элемент Canvas, который позволяет рисовать графические примитивы. Каждая клетка представлена прямоугольником, цвет которого зависит от её состояния: белый — «мертвая» клетка, зелёный — «живая».

Размер сетки динамически изменяется при изменении размеров окна. При масштабировании окна ячейки автоматически перестраиваются, сохраняя пропорции.

Перед запуском пользователю предлагается выбрать способ задания начального состояния:

- ручной ввод — с помощью кликов мышью по ячейкам; отмеченные клетки становятся зелёными;
- случайная генерация — по кнопке «Случайно», с вероятностью 0.3 для каждой клетки быть живой.

Начальные значения по умолчанию:

- размер сетки — 15 ячеек по каждой стороне;
- количество итераций — 100 шагов.

Для обоих параметров предусмотрены проверки корректности ввода. При попытке задать значение вне допустимого диапазона (размер от 1 до 50, итерации от 1 до 500) выводится сообщение об ошибке с помощью диалогового окна messagebox. При запуске работы автомата (режим running) происходит постепенное обновление состояния клеток. Обновление экрана выполняется асинхронно с помощью метода after, что позволяет сохранить отзывчивость интерфейса во время анимации.

Для предотвращения некорректных действий реализованы три режима интерфейса:

1. idle — исходное состояние перед запуском. Доступны все элементы управления, кроме кнопки паузы;
2. running — автомат в процессе работы. Недоступно изменение размера поля, количества итераций и ручное редактирование клеток;

3. `paused` — автомат находится на паузе. Кнопка «Пауза» меняет текст на «Продолжить», при нажатии выполнение возобновляется с прерванной итерации.

При очистке поля кнопкой «Очистить» эволюция автомата прекращается, счётчик итераций сбрасывается, а поле становится пустым. При повторном запуске в качестве начальных условий используется текущее состояние сетки.

Визуальное обновление клеток производится функцией `update_display`, которая проходится по всем ячейкам и устанавливает цвет в соответствии с текущим состоянием. При клике по ячейке вызывается метод `toggle_cell`, который инвертирует её состояние и обновляет отображение.

3 Анализ клеточного автомата

В ходе анализа клеточного автомата были проведены многократные симуляции приложения на 20-50 шагов со случайным заполнением «живыми» клетками с плотностью 0.5. Размеры поля задавались различные.

Для заданного правила не обнаружено устойчивых паттернов.

При текущей реализации автомат сходится ни к пустому состоянию в среднем за 5 итераций (рисунки 3а - 3f).

Для классификации по типам поведения был проведен ряд статистических исследований (рисунок 2). Значения энтропии колеблются в среднем между 0.3-0, то есть система стремится к нулю.

На основе собранных статистических данных и визуального наблюдения автомат можно отнести к 1 классу поведения по Вольфраму, так как он быстро переходит в устойчивое состояние, где все клетки мертвые.

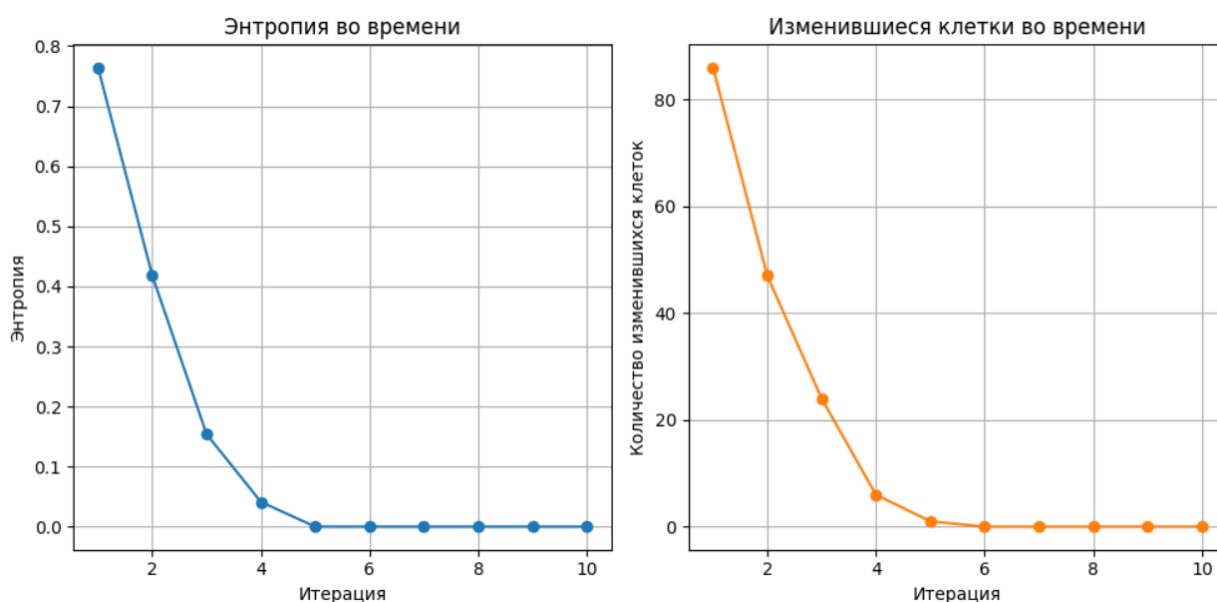
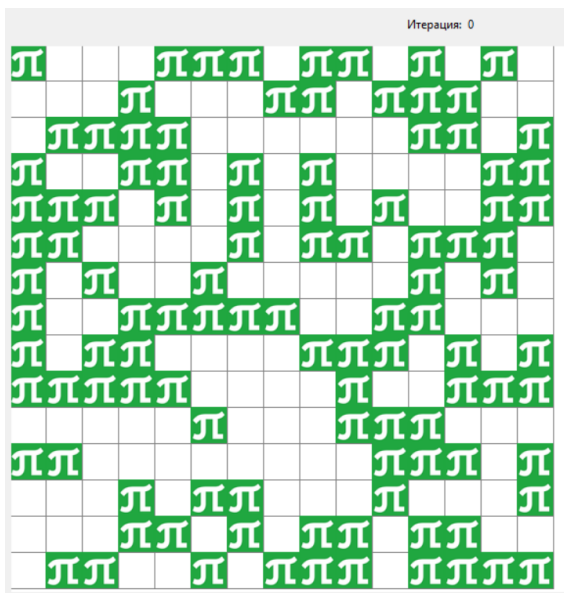
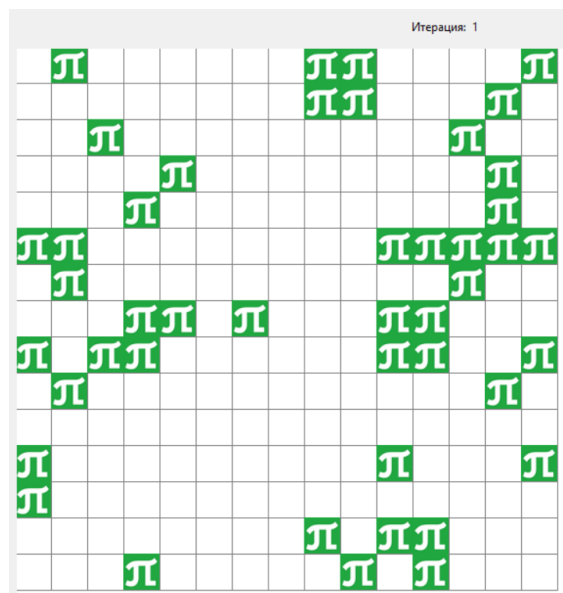


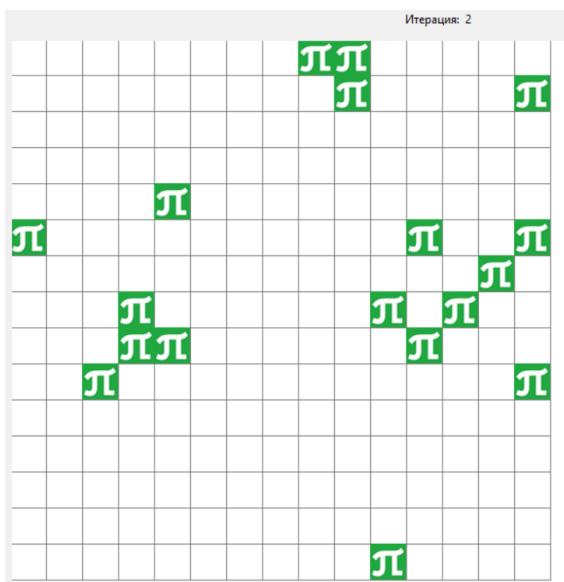
Рис. 2. Статистика: энтропия и количество изменившихся клеток



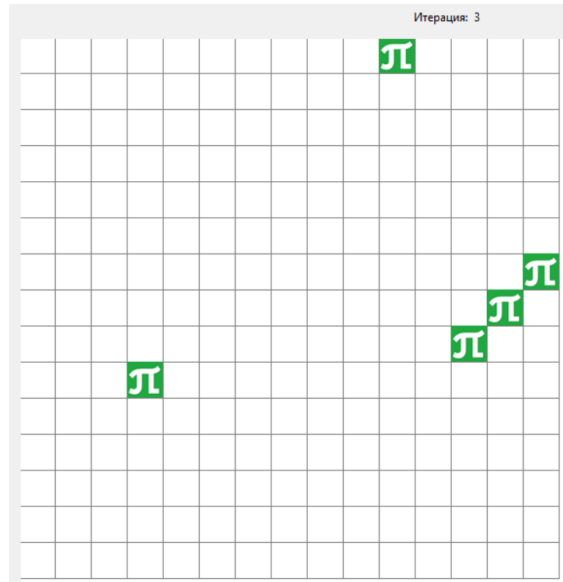
(a) 0 итерация работы автомата



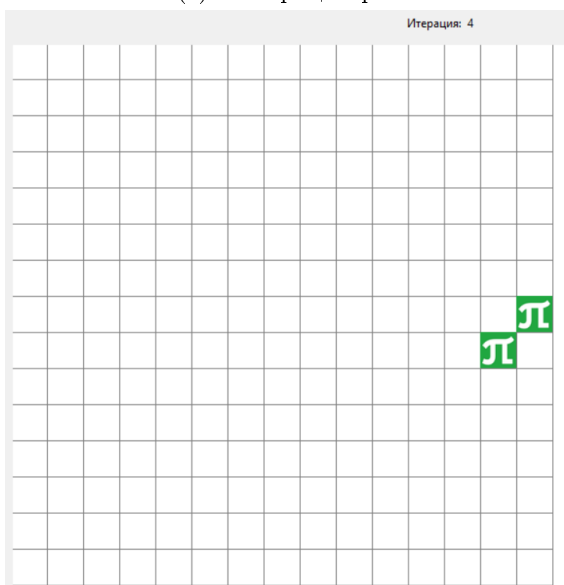
(b) 1 итерация работы автомата



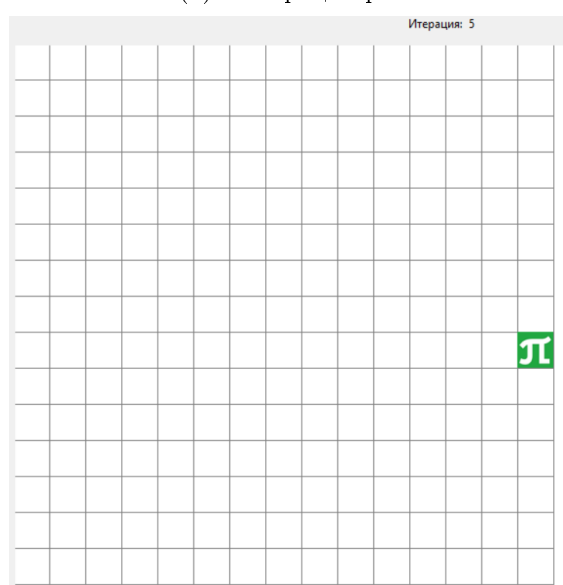
(c) 2 итерация работы автомата



(d) 3 итерация работы автомата



(e) 4 итерация работы автомата



(f) 5 итерация работы автомата

4 Результаты работы программы

В начале работы программы представлено на рисунке 4. Пользователь может выбрать тип заполнения: ручной (рисунок 5) или рандомный по кнопке «Случайно» (рисунок 6).

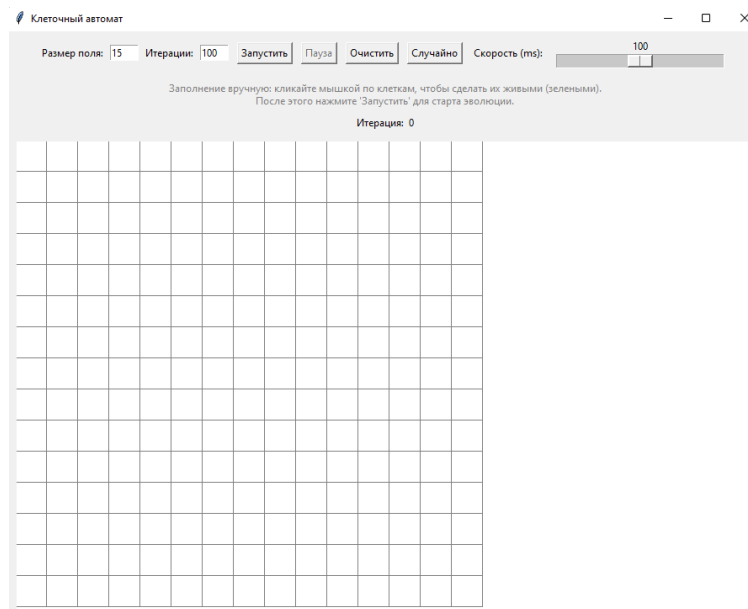


Рис. 4. Начало работы программы

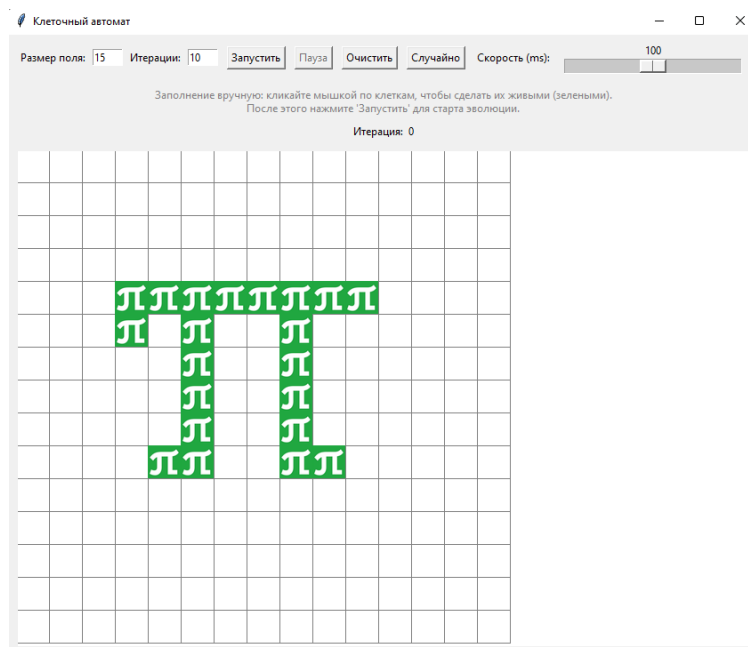


Рис. 5. Ручной ввод

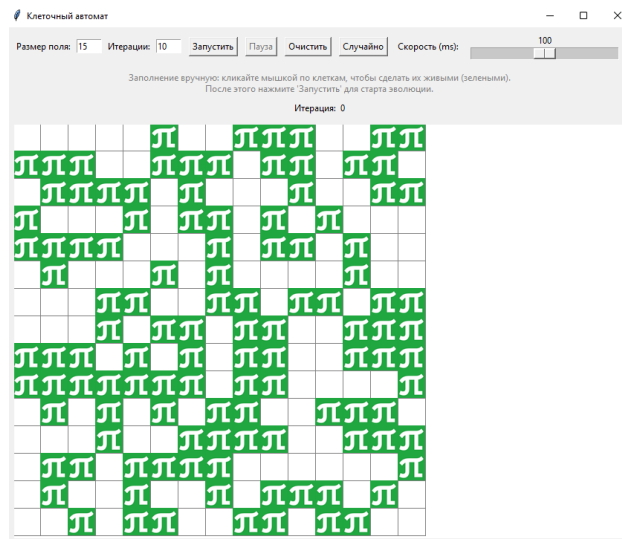


Рис. 6. Случайный ввод

При неправильном заполнении полей или попытке запустить автомат на пустой сетке выводятся сообщения об ошибке (рисунки 7 - 9)

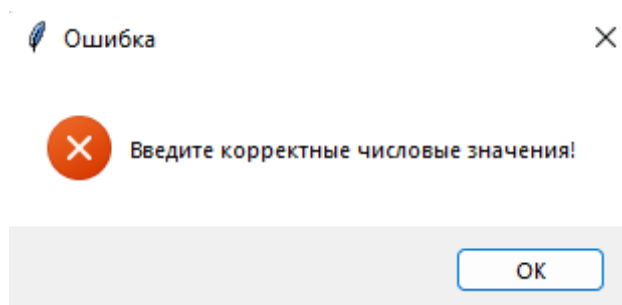


Рис. 7. Ошибка количества итераций

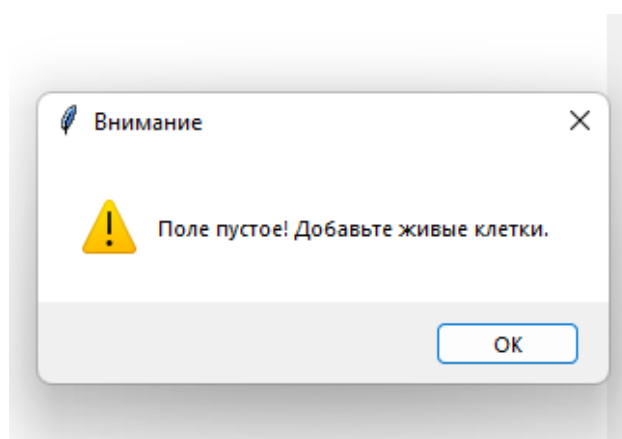


Рис. 8. Ошибка пустого поля при запуске

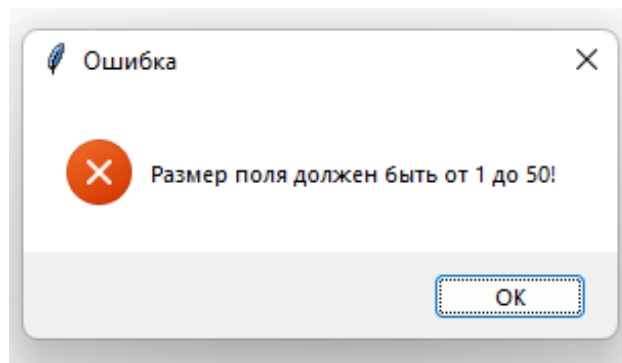
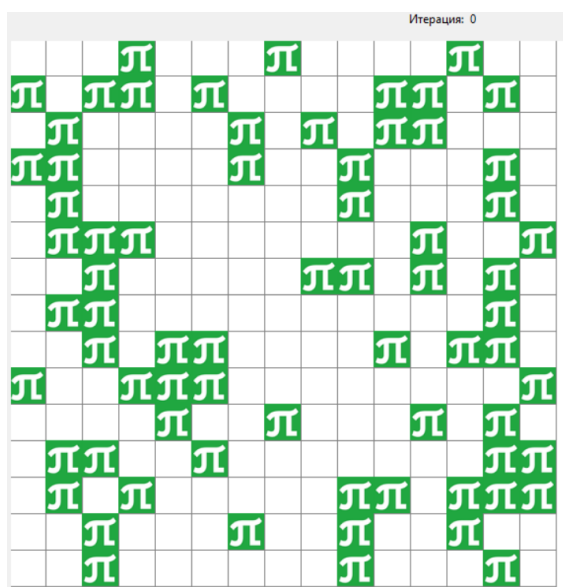
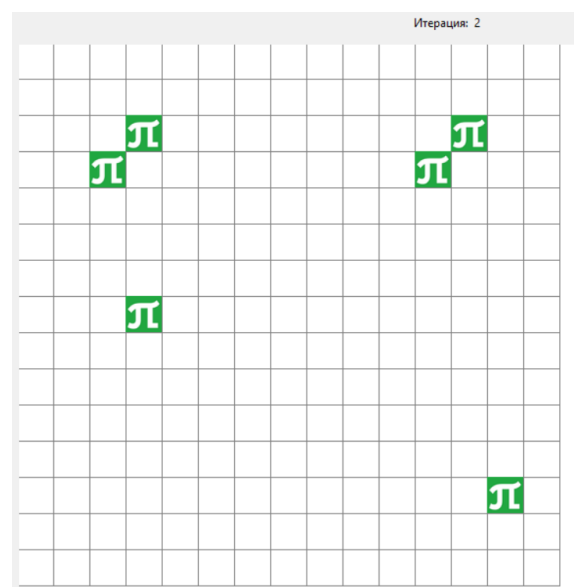


Рис. 9. Ошибка размерности поля

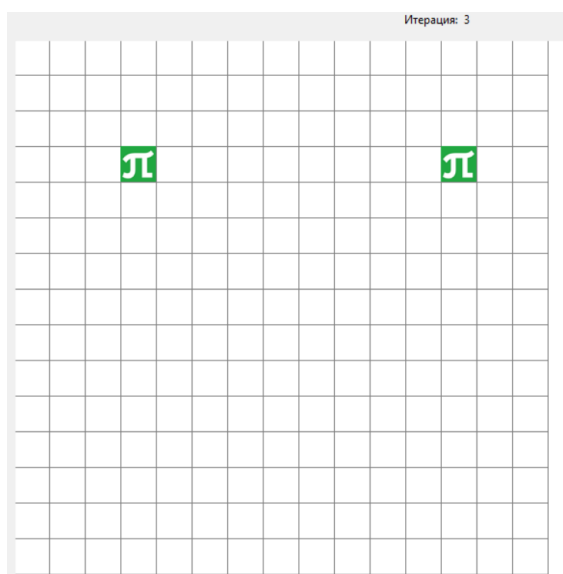
Примеры работы автомата на разных итерациях представлены на рисунках 10а - 10d.



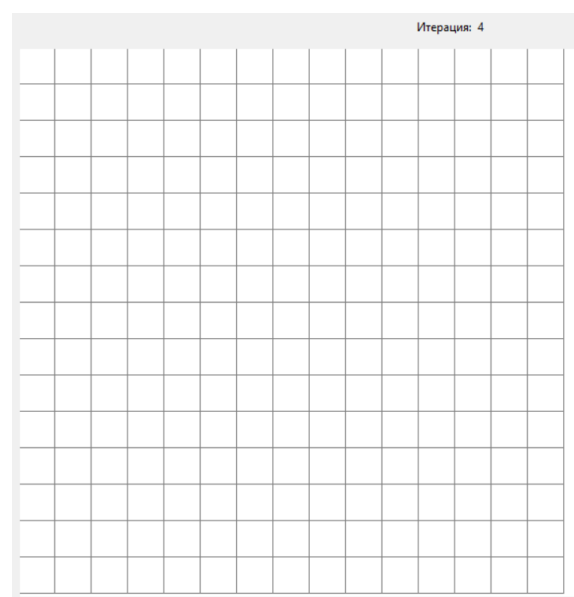
(a) 0 итерация работы автомата



(b) 2 итерация работы автомата



(c) 3 итерация работы автомата



(d) 4 итерация работы автомата

Рис. 10. Эволюция клеточного автомата на различных итерациях

Заключение

Результатом проделанной работы стала программа, которая реализует двумерный клеточный автомат с окрестностью фон Неймана и тороидальными граничными условиями. Автомат работает по заданному правилу $N = 1323300_{10} = 101000011000100100100_2$. Пользователь может задавать ширину поля и количество итераций. Программа учитывает возможность ввода как вручную (с помощью кликов по ячейкам), так и случайным образом с плотностью живых клеток равной 0.5. Клеточный автомат реализован в графике с помощью библиотеки Tkinter в Python. В ходе анализа автомата было установлено, что он относится к первому классу поведения по Вольфраму, так как он быстро сходится к устойчивому состоянию.

Достоинства программы:

- программа имеет графическое представление, что позволяет наглядно оценить эволюцию автомата;
- формирование индекса состояния клетки с помощью побитовых сдвигов ускоряет вычисления и позволяет легко адаптировать код при изменении числа соседей;
- код структурирован по классам, что улучшает читаемость.

Недостатки программы:

- ограничена производительность при размерах поля свыше 50x50 ячеек из-за однопоточной обработки и покадровой перерисовки интерфейса;
- отсутствует возможность сохранения и загрузки состояния поля.

Масштабирование программы:

- добавить пользователю возможность ввода нового правила во время работы приложения;
- расширить радиус окрестность фон Неймана до второго порядка (учитывать клетки, находящиеся на расстоянии двух шагов);
- реализовать сохранение конфигураций поля на конкретной итерации в файл.

Список использованной литературы

- [1] Секция «Телематика». — Текст : электронный // tema.spbstu.ru : [сайт]. URL: <https://tema.spbstu.ru/algorithm/> (дата обращения: 07.10.2025).
- [2] Клеточные автоматы. Реализация и эксперименты [Электронный ресурс]. — URL: <http://www.softcraft.ru/auto/switch/kla/article.pdf> (дата обращения: 07.10.2025).
- [3] Клеточный автомат // Википедия URL: Клеточный автомат — Википедия (дата обращения: 07.10.2025).
- [4] Обзор свойств клеточных автоматов, их применения». Сетевое научное издание «Системный анализ в науке и образовании», 2017. 11 стр.
- [5] Курс: Теория клеточных автоматов [Электронный ресурс]. — URL: https://apps.openedu.ru/learning/course/course-v1:merphi+cel_auto+fall_2025/home (дата обращения: 07.10.2025).

Приложение А «Графическая часть приложения»

Листинг 6. Графическая часть приложения

```
1 class AutomatonApp:
2     def __init__(self, master):
3         self.master = master
4         self.master.title("Клеточный автомат")
5         pygame.mixer.init()
6
7         self.images = {}
8         self.use_images = False
9         self.image_items = []
10
11        self.master.geometry("900x700")
12        self.running = False
13        self.ca = None
14        self.cells = []
15        self.cell_size = 20
16        self.editable = True
17
18        self.load_images()
19
20        self.entropy_values = []
21        self.diff_values = []
22        self.iterations = []
23
24        # панель управления
25        control_frame = tk.Frame(master)
26        control_frame.pack(pady=5)
27
28        tk.Label(control_frame, text="Размер поля:").grid(row=0, column=0)
29        self.size_entry = tk.Entry(control_frame, width=5)
30        self.size_entry.insert(0, "15")
31        self.size_entry.grid(row=0, column=1, padx=5)
32        self.size_entry.bind("<Return>", self.on_size_change)
33
34        tk.Label(control_frame, text="Итерации:").grid(row=0, column=2)
35        self.iter_entry = tk.Entry(control_frame, width=5)
36        self.iter_entry.insert(0, "10")
37        self.iter_entry.grid(row=0, column=3, padx=5)
38
39        self.start_button = tk.Button(control_frame, text="Запустить", command=self.start)
40        self.start_button.grid(row=0, column=4, padx=5)
41
42        self.pause_button = tk.Button(control_frame, text="Пауза", command=self.toggle_pause, state="disabled")
43        self.pause_button.grid(row=0, column=5, padx=5)
44
45        self.clear_button = tk.Button(control_frame, text="Очистить", command=self.clear)
46        self.clear_button.grid(row=0, column=6, padx=5)
47
48        self.random_button = tk.Button(control_frame, text="Случайно", command=self.randomize)
49        self.random_button.grid(row=0, column=7, padx=5)
50
```

```

51 tk.Label(control_frame, text="Скорость (ms):").grid(row=0, column=8, padx
    =5)
52 self.speed_scale = tk.Scale(control_frame, from_=0.1, to=200, orient="
    horizontal", length=200)
53 self.speed_scale.set(100)
54 self.speed_scale.grid(row=0, column=9, padx=5)
55
56 # Подсказка
57 info = ("Заполнение вручную: кликайте мышкой по клеткам, чтобы сделать их живыми
    зелеными().\n"
58 "После этого нажмите Запустить'' для старта эволюции.")
59 self.info_label = tk.Label(master, text=info, justify="center", fg="gray")
60 self.info_label.pack(pady=5)
61
62 iter_frame = tk.Frame(master)
63 iter_frame.pack()
64 tk.Label(iter_frame, text="Итерация:").pack(side="left")
65 self.iter_label = tk.Label(iter_frame, text="0")
66 self.iter_label.pack(side="left")
67 self.current_step = 0
68 self.total_steps = 0
69
70 # Холст
71 self.canvas = tk.Canvas(master, bg="white")
72 self.canvas.pack(fill="both", expand=True, padx=10, pady=10)
73 self.canvas.bind("<Configure>", self.on_resize)
74 self.canvas.bind("<Button-1>", self.toggle_cell)
75
76 # Изначальная отрисовка сетки
77 self.master.after(100, self.initial_setup)
78
79 def initial_setup(self):
80     """Первоначальная настройка сетки"""
81     try:
82         n = int(self.size_entry.get())
83         self.setup_grid(n)
84     except ValueError:
85         pass
86
87 def load_images(self):
88     try:
89         image = Image.open("logo.png")
90         image = image.resize((self.cell_size, self.cell_size), Image.Resampling.
            LANCZOS)
91         self.images[1] = ImageTk.PhotoImage(image)
92         self.use_images = True
93     except Exception as e:
94         print("Не удалось загрузить изображение, используем цвета:", e)
95         self.use_images = False
96
97 def on_size_change(self, event):
98     """Обработка изменения размера поля"""
99     if not self.running and self.editable:
100     try:
101         n = int(self.size_entry.get())
102         if n <= 0 or n > 50:
103             messagebox.showerror("Ошибка", "Размер поля должен быть от 1 до 50!")
104             return
105         self.setup_grid(n)
106     except ValueError:

```

```

107 messagebox.showerror("Ошибка", "Введите корректное число!")
108
109 # Методы графики rip
110 def setup_grid(self, n):
111     self.ca = CellularAutomaton(n)
112     if self.canvas.winfo_width() > 1 and self.canvas.winfo_height() > 1:
113         self.cell_size = min(self.canvas.winfo_width() // n, self.canvas.
            winfo_height() // n)
114     else:
115         self.cell_size = 20
116     if self.use_images:
117         self.resize_image()
118     for row in self.cells:
119         for cell in row:
120             self.canvas.delete(cell)
121         for item in getattr(self, "image_items", []):
122             self.canvas.delete(item)
123         self.cells = []
124         self.image_items = []
125
126     for r in range(n):
127         row_cells = []
128         for c in range(n):
129             cell = self.canvas.create_rectangle(
130                 c * self.cell_size, r * self.cell_size,
131                 (c + 1) * self.cell_size, (r + 1) * self.cell_size,
132                 fill="white", outline="gray")
133             row_cells.append(cell)
134         self.cells.append(row_cells)
135     def on_resize(self, event):
136         if self.ca:
137             self.redraw_grid()
138
139     def redraw_grid(self):
140         n = self.ca.n
141         if self.canvas.winfo_width() > 1 and self.canvas.winfo_height() > 1:
142             self.cell_size = min(self.canvas.winfo_width() // n, self.canvas.
                winfo_height() // n)
143         if self.use_images:
144             self.resize_image()
145
146         for r in range(n):
147             for c in range(n):
148                 x1 = c * self.cell_size
149                 y1 = r * self.cell_size
150                 x2 = x1 + self.cell_size
151                 y2 = y1 + self.cell_size
152                 self.canvas.coords(self.cells[r][c], x1, y1, x2, y2)
153                 color = self.get_color(self.ca.grid[r][c])
154                 if color == "IMAGE":
155                     self.canvas.itemconfig(self.cells[r][c], fill="white")
156                 else:
157                     self.canvas.itemconfig(self.cells[r][c], fill=color)
158             self.update_display()
159
160     def get_color(self, state):
161         if self.use_images and state == 1:
162             return "IMAGE"
163         else:
164             return {0: "white", 1: "#22aa22"}.get(state, "white")

```

```

165
166 def toggle_cell(self, event):
167 if not self.ca or not self.editable or self.running:
168 return
169 c = event.x // self.cell_size
170 r = event.y // self.cell_size
171 if 0 <= r < self.ca.n and 0 <= c < self.ca.n:
172 self.ca.grid[r][c] = 1 - self.ca.grid[r][c]
173 self.update_display()
174
175 def resize_image(self):
176 """Масштабирует изображение под текущий размер клетки"""
177 try:
178 original = Image.open("logo.png")
179 resized = original.resize((max(1, self.cell_size), max(1, self.cell_size))
180 , Image.Resampling.LANCZOS)
181 self.images[1] = ImageTk.PhotoImage(resized)
182 self.use_images = True
183 except Exception as e:
184 print("Ошибка при масштабировании изображения:", e)
185 self.use_images = False
186
187 def update_display(self):
188 for item in getattr(self, "image_items", []):
189 try:
190 self.canvas.delete(item)
191 except Exception:
192 pass
193 self.image_items = []
194 for r in range(self.ca.n):
195 for c in range(self.ca.n):
196 if self.use_images and self.ca.grid[r][c] == 1:
197 x = c * self.cell_size + self.cell_size // 2
198 y = r * self.cell_size + self.cell_size // 2
199 img_id = self.canvas.create_image(x, y, image=self.images[1], anchor="
200 center")
201 self.image_items.append(img_id)
202 self.canvas.itemconfig(self.cells[r][c], fill="white")
203 else:
204 color = self.get_color(self.ca.grid[r][c])
205 self.canvas.itemconfig(self.cells[r][c], fill=color)
206 def set_controls_state(self, state):
207 """Установка состояния элементов управления"""
208 if state == 'idle':
209 # До начала выполнения
210 self.size_entry.config(state="normal")
211 self.iter_entry.config(state="normal")
212 self.random_button.config(state="normal")
213 self.start_button.config(state="normal")
214 self.pause_button.config(state="disabled")
215 self.pause_button.config(text="Пауза")
216 self.editable = True
217
218 elif state == 'running':
219 # Во время выполнения
220 self.size_entry.config(state="disabled")
221 self.iter_entry.config(state="disabled")
222 self.random_button.config(state="disabled")
223 self.start_button.config(state="disabled")
224 self.pause_button.config(state="normal")

```

```

223 self.pause_button.config(text="Пауза")
224 self.editable = False
225
226 elif state == 'paused':
227     # Во время паузы
228     self.size_entry.config(state="disabled")
229     self.iter_entry.config(state="disabled")
230     self.random_button.config(state="disabled")
231     self.start_button.config(state="disabled")
232     self.pause_button.config(state="normal")
233     self.pause_button.config(text="Продолжить")
234     self.editable = False
235
236 def start(self):
237     try:
238         n = int(self.size_entry.get())
239         self.total_steps = int(self.iter_entry.get())
240
241         if n <= 0 or n > 50:
242             messagebox.showerror("Ошибка", "Размер поля должен быть от 1 до 50!")
243             return
244         if self.total_steps <= 0 or self.total_steps > 1000000:
245             messagebox.showerror("Ошибка", "Количество итераций должно быть от 1 до 1000000!")
246             return
247
248         except ValueError:
249             messagebox.showerror("Ошибка", "Введите корректные числовые значения!")
250             return
251
252         if not self.ca:
253             self.setup_grid(n)
254         else:
255             self.update_display()
256
257         if all(all(v == 0 for v in row) for row in self.ca.grid):
258             messagebox.showwarning("Внимание", "Поле пустое! Добавьте живые клетки.")
259             return
260
261         try:
262             pygame.mixer.music.load("mediv.mp3")
263             pygame.mixer.music.play(-1)
264             pygame.mixer.music.set_volume(0.5)
265         except Exception as e:
266             print("Error loading music:", e)
267
268         self.running = True
269         self.current_step = 0
270         self.iter_label.config(text="0")
271         self.set_controls_state('running')
272         self.run_step()
273
274         def run_step(self):
275             if not self.running:
276                 return
277
278             if self.current_step >= self.total_steps:
279                 self.running = False
280                 self.set_controls_state('idle')
281                 pygame.mixer.music.stop()

```



```

282 #self.show_graphs()
283 return
284
285 old_grid = [row[:] for row in self.ca.grid]
286 self.ca.step()
287 self.update_display()
288 self.current_step += 1
289 self.iter_label.config(text=str(self.current_step))
290
291 entropy = self.ca.measure_entropy()
292 diff = self.ca.hamming_distance(old_grid)
293
294 self.iterations.append(self.current_step)
295 self.entropy_values.append(entropy)
296 self.diff_values.append(diff)
297
298 delay = int(201 - self.speed_scale.get())
299 speed = self.speed_scale.get()
300 volume = min(max(speed / 200, 0.2), 1.0)
301 pygame.mixer.music.set_volume(volume)
302
303 self.master.after(delay, self.run_step)
304
305 def set_volume(self, volume_factor):
306     """Изменяет громкость музыки (0.0 - 1.0)"""
307     try:
308         if not pygame.mixer.get_init():
309             pygame.mixer.init()
310             pygame.mixer.music.set_volume(volume_factor)
311         except Exception as e:
312             print("Error setting volume:", e)
313
314     def show_graphs(self):
315         """Построение графиков энтропии, числа изменившихся и живых клеток"""
316         if not self.iterations:
317             return
318
319         plt.close('all')
320         if len(self.iterations) > 50:
321             step = 10
322             iters = self.iterations[::step]
323             entropy = self.entropy_values[::step]
324             diff = self.diff_values[::step]
325         else:
326             iters = self.iterations
327             entropy = self.entropy_values
328             diff = self.diff_values
329         total_cells = self.ca.n * self.ca.n
330
331         plt.figure(figsize=(15, 5))
332
333         plt.subplot(1, 3, 1)
334         plt.plot(iters, entropy, marker='o', color='tab:blue')
335         plt.title("Энтропия во времени")
336         plt.xlabel("Итерация")
337         plt.ylabel("Энтропия")
338         plt.grid(True)
339
340         plt.subplot(1, 3, 2)
341         plt.plot(iters, diff, marker='o', color='tab:orange')

```

```

342 plt.title("Изменившиеся клетки во времени")
343 plt.xlabel("Итерация")
344 plt.ylabel("Количество изменившихся клеток")
345 plt.grid(True)
346
347 plt.tight_layout()
348 plt.show()
349
350 self.iterations.clear()
351 self.entropy_values.clear()
352 self.diff_values.clear()
353
354 def toggle_pause(self):
355     if self.running:
356         self.running = False
357         self.set_controls_state('paused')
358     try:
359         pygame.mixer.music.pause() # приостанавливаем музыку
360     except Exception as e:
361         print("Error pausing music:", e)
362     else:
363         self.running = True
364         self.set_controls_state('running')
365     try:
366         pygame.mixer.music.unpause() # продолжаем с того же места
367     except Exception as e:
368         print("Error resuming music:", e)
369     self.run_step()
370
371 def clear(self):
372     self.running = False
373     self.current_step = 0
374     self.iter_label.config(text="0")
375     try:
376         pygame.mixer.music.pause() # приостанавливаем музыку
377     except Exception as e:
378         print("Error pausing music:", e)
379     if not self.ca:
380     try:
381         n = int(self.size_entry.get())
382         if 1 <= n <= 50:
383             self.setup_grid(n)
384         except ValueError:
385             pass
386     else:
387         self.ca.clear()
388         self.update_display()
389         self.set_controls_state('idle')
390
391 def randomize(self):
392     if self.running:
393         return
394     if not self.ca:
395     try:
396         n = int(self.size_entry.get())
397         if n <= 0 or n > 50:
398             messagebox.showerror("Ошибка", "Размер поля должен быть от 1 до 50!")
399         return
400         self.setup_grid(n)
401     except ValueError:

```

```
402 messagebox.showerror("Ошибка", "Введите корректный размер поля!")
403 return
404 self.ca.randomize()
405 self.update_display()
406
407
408
```